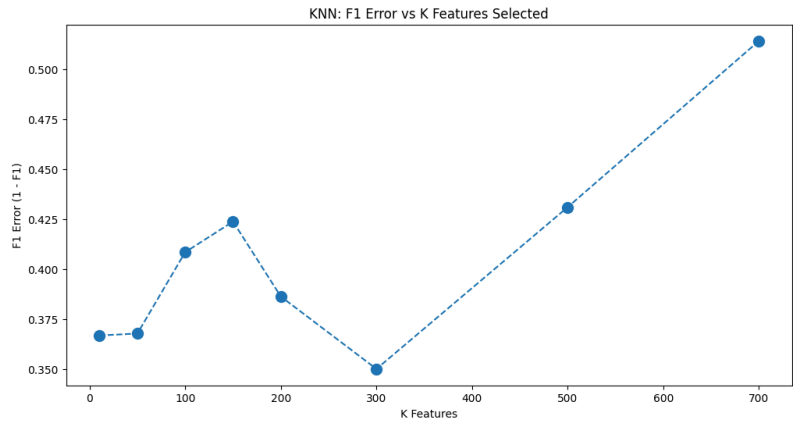


Username: vnguy7
F1 score: 0.75

Before everything, I ran a default run just to know every classifier most comfortable k range (for SelectKBest) to narrow things down for each classifier, putting the k-range to be [10, 50, 100, 150, 200, 300, 500, 700]
Turns out the best k value for selectKBest is:

KNN:
After the initial run



```
KNN
Testing k=10... F1: 0.633 (std: 0.051)
Testing k=50... F1: 0.632 (std: 0.043)
Testing k=100... F1: 0.592 (std: 0.045)
Testing k=150... F1: 0.576 (std: 0.062)
Testing k=200... F1: 0.614 (std: 0.040)
Testing k=300... F1: 0.650 (std: 0.049)
Testing k=500... F1: 0.569 (std: 0.039)
Testing k=700... F1: 0.486 (std: 0.027)
Best k (features) for KNN: 300, F1: 0.650
Decision Tree
Testing k=10... F1: 0.499 (std: 0.087)
Testing k=50... F1: 0.544 (std: 0.040)
Testing k=100... F1: 0.561 (std: 0.059)
Testing k=150... F1: 0.514 (std: 0.053)
Testing k=200... F1: 0.597 (std: 0.046)
Testing k=300... F1: 0.514 (std: 0.078)
Testing k=500... F1: 0.564 (std: 0.081)
Testing k=700... F1: 0.554 (std: 0.051)
Best k (features) for DT: 200, F1: 0.597
Naive Bayes
Testing k=10... F1: 0.672 (std: 0.036)
Testing k=50... F1: 0.686 (std: 0.042)
Testing k=100... F1: 0.681 (std: 0.047)
Testing k=150... F1: 0.698 (std: 0.044)
Testing k=200... F1: 0.688 (std: 0.036)
Testing k=300... F1: 0.684 (std: 0.055)
Testing k=500... F1: 0.637 (std: 0.060)
Testing k=700... F1: 0.589 (std: 0.075)
Best k (features) for NB: 150, F1: 0.698
```

I decided to try the k range (Select K Best) of [15, 35, 50, 100, 300, 500, 550] and the k range of the number of neighbor values in the KNN model of [1, 3, 5, 9, 11, 15, 19, 23] while still using the default.

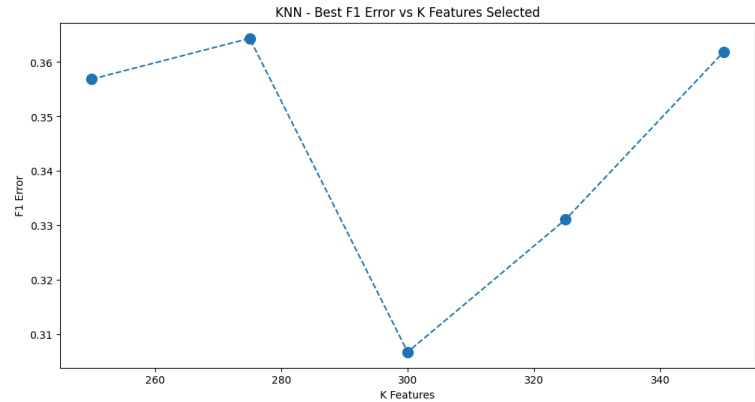
Best config:
Features k = 300, Neighbors = 3
Best F1 = 0.693

The result shows the best k is 300, and the best neighbor value is 3

So I decided to test different nearest neighbor values and metric values while keeping SelectKBest to be around 300, and chi2 (mutual information took too long to run):

- k_values = [250, 275, 300, 325, 350]
- neighbor_values = [1, 3, 5, 9, 11, 15, 19, 23]
- metric_values = ['euclidean', 'manhattan', 'cosine'] // For the cosine case, I will normalize the data for it, but not for the euclidean or manhattan

Turns out the old config was still the best one (and Euclidean is by far the best metric).



Best config:
Features k = 300
Neighbors = 3
Metric = euclidean
Best F1 = 0.693

Here's the table where the SelectKBest value was 300

n_neighbors	euclidean	cosine	manhattan
1	0.69	0.342	0.295
3	0.693	0.447	0.315
5	0.65	0.493	0.314
9	0.665	0.522	0.286
11	0.669	0.501	0.27
15	0.664	0.519	0.254
19	0.659	0.495	0.238

23	0.639	0.481	0.226
----	-------	-------	-------

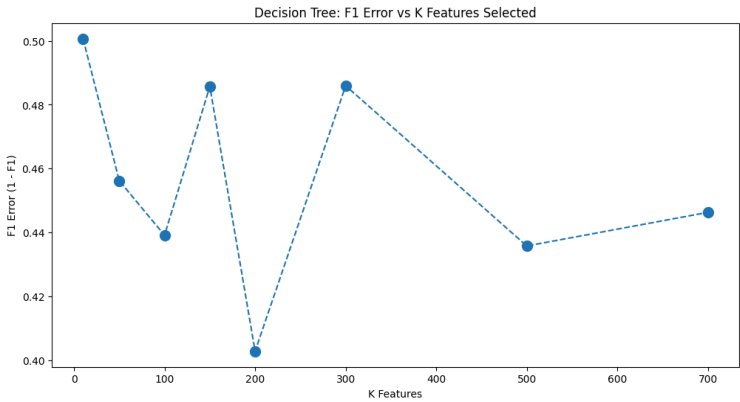
Decide this is the classification model used for KNN:

- N_neighbors = 3
- K = 300
- Score_func = chi2
- Metric = euclidean

Run time is very good, around 5 min for the whole set above

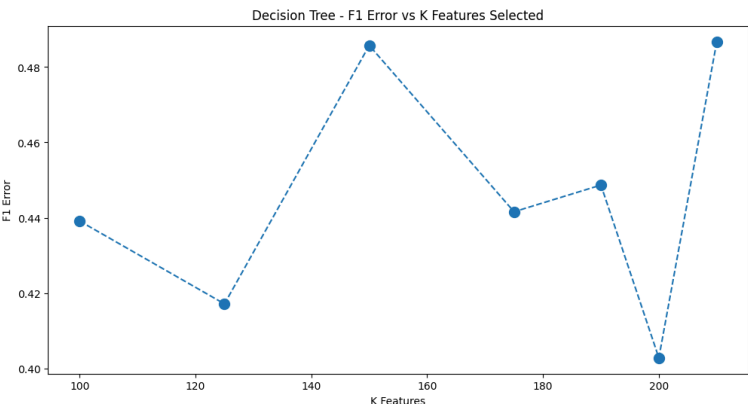
Decision Tree:

After the initial run



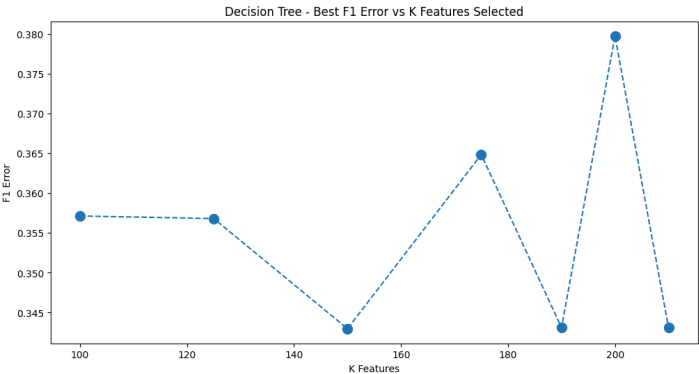
I decided to test the selectKBest range around [100, 125, 150, 175, 190, 200, 210]
 The best result came from k = 200, but the result is pretty bad

Best config:
 Features k = 200
 Best F1 = 0.597



So I decided it's time to adjust the max depth of the tree (should change the result dramatically) and try out all of the impurity measurement methods: Gini, entropy, log_loss. While keeping the Select K best to be [100, 125, 175, 190, 200, 210]

The best result is deemed to be:



Best config:
 Features k = 150
 Max depth = 3
 Criterion = entropy
 Best F1 = 0.657

So k is 150 for SelectbestK in this table

max_depth	Gini	Entropy	Log_loss
1	0.620	0.620	0.620

2	0.635	0.635	0.635
3	0.594	0.657	0.657
4	0.566	0.627	0.627

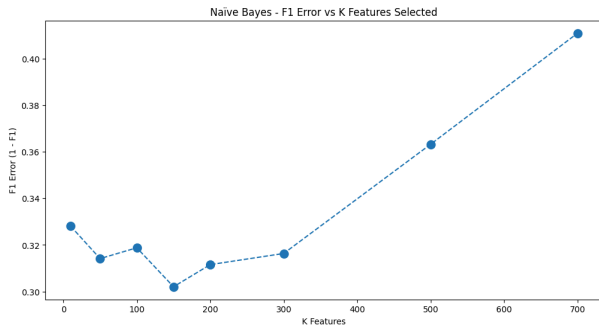
Decide this is the classification model used for the Decision Tree:

- Criterion = entropy
- SelectbestK k = 150
- Score_func = chi2
- Max_depth = 3

Run time is very good, around 5 min

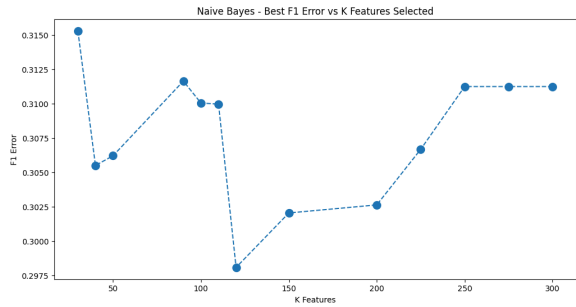
Naive Bayes:

After the initial run



I decided to test the k range of SelectBestK to be [30, 40, 50, 90, 100, 110, 120, 150, 200, 225, 250, 275, 300] and since this is the most promising data i also decided to test the alpha range to be [0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0]

The data is very promising:

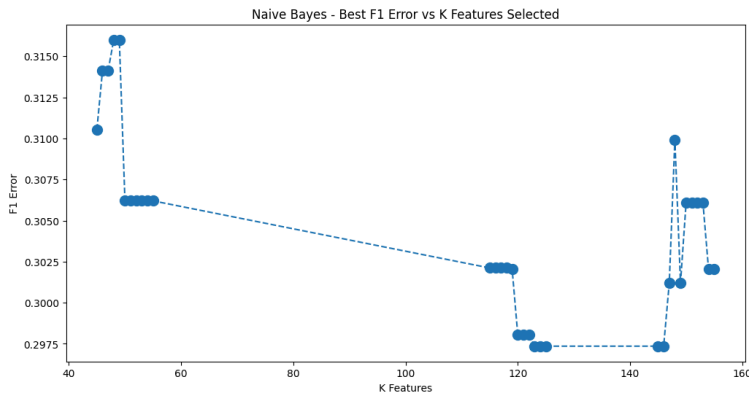


I decided to focus on 3 regions: around 50, around 120, and around 150:

[45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155]

And drop 1.0, 2.0, and 5.0 from the alpha list

The best result in predicting is:



Best config:
Features k = 123
Alpha = 0.001
Best F1 = 0.703

But the standard deviation of the above is 0.06, which makes the prediction unstable, so more candidates are fit to be used for prediction:

- SelectKBest k = 50, alpha = 0.001 - F1 score: 0.694, standard deviation of 0.046
- SelectKBest k = 120, alpha = 0.001 - F1 score: 0.702, standard deviation of 0.057
- SelectKBest k = 155, alpha = 0.5 - F1 score: 0.698, standard deviation of 0.044
- SelectKBest k = 123, alpha = 0.001 - F1 score: 0.703, standard deviation of 0.06

The table here is for SelectKBest k = 155

alpha	F-Score
0.001	0.690
0.01	0.690
0.1	0.690
0.5	0.698

Out of all of these, since the difference in the F1 score is very minimal, I chose to go with the one with best standard deviation

Naive Bayes:

- K = 155
- Alpha = 0.5

Run time is average, around 15 min for the whole set above, but it's due to the fact that I chose to test a lot of stuff at the same time

Conclusion:

	F1 score
KNN (n_neighbors = 3, metric='euclidean') - SelectKBest = 300	0.693
Decision Tree (criterion = 'entropy', max_depth = 20) - k fold =	0.657
Naïve Bayes (default parameter, BernoulliNB(alpha=0.001)) - k fold = 53	0.698

Choose Naive Bayes as the final classifier to make predictions on the data.